

Genetic Programming Hyper-Heuristics for the Job Shop Scheduling Problem with Interval Durations: Robustness-Aware Interpretable Rules that Generalize Across Instance Sizes

Hernán Díaz Rodríguez

Department of Computing, University of Oviedo, Campus de Gijón, Gijón, 33204, Spain

Abstract

Automated dispatching-rule design has not been explored for the job shop scheduling problem with interval durations (IJSP), where processing times are known only to lie within intervals and schedules are built by interval arithmetic. We present a genetic programming hyper-heuristic that evolves priority rules over an *interval-aware* terminal set, exposing both the worst-case values and the widths of the uncertain quantities, and deploys them in a single constructive pass. On the 70 interval Taillard instances, 30 evolutions average 18.99% (± 1.33) relative error and the best rule attains 17.71%, improving on the strongest constructive baseline, Giffler–Thompson/MWKR (29.5%), on every instance. Evolved on four 20×15 instances, the rules transfer zero-shot to all size classes and to a second benchmark, remain human-readable, and are insensitive to the choice of training instances. A reimplementa-tion of a published genetic algorithm, sharing the rule’s decoder, needs some 7.6×10^4 schedule constructions to match one pass of the evolved rule, and gains markedly from being seeded with it. Ablations

Email address: `diazhernan@uniovi.es` (Hernán Díaz Rodríguez)

delimit what interval awareness contributes. Under the default expected-makespan objective it is unnecessary: removing the width terminals, or evolving on crisp midpoints, does equally well. Its value lies in robustness: an objective penalizing interval width yields narrower predicted intervals and reduces the deviation of the executed makespan by about one fifth, a gain that requires the width terminals and persists under non-uniform and worst-case realizations and on asymmetric intervals, at the cost of a higher expected makespan.

Keywords: interval job shop scheduling, genetic programming, hyper-heuristics, dispatching rules, scheduling under uncertainty, robustness

1. Introduction

In real manufacturing environments, the exact duration of an operation is rarely known in advance (Allahverdi, 2022). The interval job shop scheduling problem (IJSP) models this uncertainty in a minimal, distribution-free way: each processing time is only assumed to lie within a known interval, and schedules are built and evaluated with interval arithmetic, so that the makespan of a solution is itself an interval and solutions are compared by an interval ranking. Population metaheuristics for the IJSP, among them genetic algorithms and artificial bee colony variants, achieve strong results at the cost of building and evaluating large numbers of complete solutions for every instance (Díaz et al., 2020, 2023b,a).

At the opposite end of the computational spectrum are *dispatching rules*: priority functions that build a schedule in a single constructive pass by repeatedly selecting the next operation to start. They combine speed, interpretability and ease of deployment. However, hand-crafted rules leave a

large quality gap with respect to search-based methods, as the experiments reported later in this paper confirm.

For *deterministic* scheduling, evolving rules automatically with genetic programming (GP) has been shown to narrow this gap substantially (Branke et al., 2016; Nguyen et al., 2017). For scheduling under interval uncertainty, however, the automated design of dispatching rules remains, to the best of our knowledge, unexplored. Moving automated rule design to interval durations raises questions that do not arise in the deterministic case: which attributes the evolution actually selects, when the quantities describing a candidate operation (its duration, its earliest start, the work remaining in its job) are intervals rather than numbers; whether the *magnitude* of the uncertainty carries information that is useful for dispatching, or whether it is the position of the intervals that drives the decision; and whether an objective that rewards a narrow makespan interval, rather than a low one, leads the evolution to schedules whose execution departs less from what was predicted. Answering these questions empirically requires exposing uncertainty to the rule as first-class attributes, which is the starting point of this work.

A last question concerns the place of such rules among the methods already available for the IJSP. Dispatching rules and per-instance metaheuristics are usually compared at their natural operating points, one constructive pass against a complete search, which ranks them but does not say for which computational budget each is preferable. Placing both on a common budget axis, within a single implementation, answers that question: up to which budget an evolved rule is the better use of computation, and what separates it from the best search methods beyond it. A further question, how interval makespans should be compared at all when they admit no natural total or-

der, we do not reopen: we adopt the ranking that a systematic study of the alternatives found most robust (Díaz et al., 2020).

This paper makes the following contributions:

1. A GP hyper-heuristic that brings automated rule design to the IJSP: priority expressions over a terminal set in which the duration, earliest start and work remaining of a candidate operation appear both as worst-case values and as widths, with fitness computed by interval arithmetic. The rules it produces are compact enough to be simplified and read by hand (Sections 4 and 7.1).
2. An empirical study of the evolved rules on the 70 interval Taillard instances comprising (i) a comparison with deterministic constructive methods at equal computational cost and (ii) an evaluation of zero-shot generalization across instance sizes and to a second benchmark of classical instances (Section 6).
3. A comparison with per-instance search, both against the published results of the IJSP metaheuristics and at matched budget against our Python implementation of the genetic algorithm published for the IJSP, in which the evolved rule, its sampled variant and the genetic algorithm, with and without the rule as a seed, share decoder and evaluator and are evaluated across budgets measured in seconds per instance, on instance sets of different sizes (Sections 6.4 and 6.5).
4. A delimitation of what interval awareness contributes, in two parts: ablations that remove the width terminals or evolve on the crisp midpoint instances with scalar fitness, under the default expected-makespan objective (Section 7.4); and an objective that penalizes interval width, which has no crisp counterpart, whose effect on the deviation of the

executed makespan from its prediction, in its mean and in its tail, is examined as a function of the weight of the penalty, under non-uniform and worst-case realizations, and on asymmetric intervals (Sections 7.5, 7.6 and 7.7).

The remainder of the paper is organized as follows. Section 2 reviews related work and Section 3 formalizes the IJSP. Section 4 presents the GP hyper-heuristic, from both the hyper-heuristic and the evolutionary viewpoints. Section 5 describes the benchmarks, the data split, the reference methods, the configuration study, the evaluation measures and the computing environment. Section 6 reports the results: evolution, the position of the evolved rules among constructive methods, generalization, their comparison with the published metaheuristics, and a comparison at matched budget with our implementation of a genetic algorithm. Section 7 analyzes them: the anatomy of the rules and a worked example, the role of the decoder, the ablations of interval awareness, the robust objective and executional robustness, and the sensitivity of the results to asymmetric intervals, before stating the limitations. Section 8 concludes.

2. Related Work

2.1. Job shop scheduling under interval uncertainty

Interval durations offer a distribution-free uncertainty model requiring only bounds, in contrast to stochastic and fuzzy approaches (Palacios et al., 2015): they are the natural choice when historical data are insufficient to estimate distributions and a decision maker can only supply a minimum and a maximum duration. Scheduling with interval processing times has been studied across machine environments, from the single machine (Allahverdi

et al., 2014) to the flexible job shop (Xu et al., 2023), with the flow shop and others surveyed by Allahverdi (2022); recent work extends the model to richer variants such as assembly (Zheng et al., 2024) and energy-aware flexible (Afşar et al., 2025) shops. It has also been studied analytically, through measures of the problem uncertainty induced by the intervals and of the stability of optimal sequences (Sotskov et al., 2013), and through the execution of schedules built under uncertain durations (Sotskov et al., 2020), the question Section 7.6 examines for evolved rules. The risk carried by a fixed schedule has also been quantified directly: Calamita et al. (2025) estimate the value-at-risk and conditional value-at-risk of its makespan when only the range of each duration is known, replacing costly exact evaluation with machine-learning predictors built on features of the activity network.

For the job shop with interval durations and makespan minimization, which concerns us here, the literature is dominated by population-based metaheuristics. Lei’s population-based neighborhood search (Lei, 2011), which ranks interval makespans by a possibility degree, was an early reference point; it was subsequently outperformed by a genetic algorithm with a systematic study of interval rankings (Díaz et al., 2020) and by artificial bee colony variants (Díaz et al., 2023b,a), which report the best published results for this problem. Related objectives have also been addressed under interval uncertainty, notably tardiness minimization (Díaz et al., 2022). All of these are iterative search methods: they build and evaluate many complete solutions and refine them over many iterations for each instance solved. Fast constructive methods for the IJSP have received far less attention, and dispatching rules with interval-aware lexicographic comparisons are the point of comparison they are measured against, the role they play in this paper (Section 6.2). Constructive heuristics are also a standard source

of initial solutions for population methods (Kazimipour et al., 2014), a use Section 6.5 examines for evolved rules.

2.2. Dispatching rules for job shop scheduling

Priority dispatching rules are among the oldest and most widely deployed scheduling heuristics. Classical rules score each ready operation by a single attribute, and we adopt their standard acronyms throughout this paper. *Shortest* and *longest processing time* (SPT, LPT) look at the duration of the operation itself. *Earliest starting time* (EST) prefers the operation that can begin soonest given the current state of its job and its machine. *Most work remaining* (MWKR) and *most operations remaining* (MOR) look instead at the job the operation belongs to, and favor the one with the largest amount of pending work, measured respectively in time units and in number of operations. The *critical ratio* (CR) weighs the work still to be done against the time available until a due date. Surveys catalog more than a hundred such variants (Panwalkar and Iskander, 1977; Haupt, 1989), and more recent ones compare those in current use across machine environments (Đurašević and Jakobović, 2018). Beyond single-attribute rules, the algorithm of Giffler and Thompson (1960), G&T in what follows, restricts each choice to a machine’s conflict set, guaranteeing *active* schedules and markedly improving plain rules when a priority criterion is used to break ties inside that set; we write G&T-SPT and G&T-MWKR for the resulting combinations. Stronger constructive procedures exist for the *deterministic* job shop, notably the shifting bottleneck heuristic (Adams et al., 1988), which repeatedly solves one-machine subproblems with heads and tails by means of Carlier’s algorithm (Carlier, 1982). These fall outside the scope of the present work: their machinery rests on exact one-machine optimization and

critical-path arguments that do not extend directly to interval arithmetic, where no natural total order is available. Hand-crafted rules, however, have reached a performance plateau: no fixed combination of attributes performs well across shop configurations and objectives (Haupt, 1989), which motivated their *automated* design.

2.3. Automated design of dispatching rules

Genetic programming (Koza, 1992; Poli et al., 2008) evolves a population of expressions, usually represented as trees, by applying selection and variation operators directly on their structure. GP hyper-heuristics apply this machinery to *priority expressions* over scheduling attributes, and constitute the standard approach to automated rule design for production scheduling (Branke et al., 2016; Nguyen et al., 2017). Evolved rules have been shown to outperform hand-crafted ones across job shop variants, with active research on surrogate fitness (Hildebrandt and Branke, 2015), feature selection (Mei et al., 2017) and multi-objective formulations (Nguyen et al., 2014). Beyond single rules, a further line combines several evolved rules into *ensembles* that jointly construct or vote on a solution (Gil-Gala et al., 2021), and recent work explores alternatives to evolutionary search for the same design task, namely systematic tree search over the space of expression trees (Gil-Gala et al., 2025b), reporting rules of comparable quality but smaller size and better readability. All of this work targets deterministic (or dynamic-stochastic) settings. Closest to our setting, Karunakaran et al. (2017) evolve rules for dynamic job shops with uncertain processing times, exposing summary statistics of observed deviations (an exponential moving average) to the rule; uncertainty there is realized stochastically during simulation, and rules are evaluated on sampled scenarios. A recent survey of

learned optimization for the job shop (Xu et al., 2025), covering both GP and DRL constructive approaches, records no work on interval-valued processing times. To the best of our knowledge, evolving rules whose *inputs* are interval attributes, and whose fitness is computed by interval arithmetic rather than by sampling realizations, has not been explored before.

Finally, deep reinforcement learning provides an alternative family of learned constructive methods for the JSP (Zhang et al., 2020; Park et al., 2021); GP rules and DRL policies are increasingly studied as competing learning paradigms for the same construction task, and controlled comparisons between them under common protocols have been identified as a gap in the recent literature (Xu et al., 2025). Both families occupy the same computational class at deployment: a single constructive pass. Reinforcement learning has recently been applied to the IJSP itself: Han et al. (2026) learn which machine-order relations to fix before execution and leave those most sensitive to interval uncertainty to be resolved during execution by timed automata, reporting lower mean midpoint makespans than population-based neighborhood search and a genetic algorithm on 14 classical instances. The rules evolved here, by contrast, fix every sequencing decision before execution.

3. The Interval Job Shop Scheduling Problem

An IJSP instance consists of n jobs and m machines. Job j is a chain of m operations $o_{j1} \prec o_{j2} \prec \cdots \prec o_{jm}$, each requiring a distinct machine $\mu(o_{jl})$ exclusively and non-preemptively. The duration of operation o is an interval $\mathbf{p}_o = [\underline{p}_o, \bar{p}_o]$ with $0 < \underline{p}_o \leq \bar{p}_o$: the only assumption is that the realized duration lies within these bounds. As in this definition, boldface denotes

an interval-valued quantity throughout, and the underlined and overlined symbols its lower and upper bounds.

Schedule construction uses interval arithmetic. For intervals $\mathbf{a} = [\underline{a}, \overline{a}]$ and $\mathbf{b} = [\underline{b}, \overline{b}]$, the two operations required by makespan minimization are

$$\mathbf{a} + \mathbf{b} = [\underline{a} + \underline{b}, \overline{a} + \overline{b}], \quad (1)$$

$$\max(\mathbf{a}, \mathbf{b}) = [\max(\underline{a}, \underline{b}), \max(\overline{a}, \overline{b})], \quad (2)$$

both component-wise, the standard operations of interval arithmetic (Moore et al., 2009) as used in the IJSP literature (Díaz et al., 2020). A solution determines a task processing order π (a permutation with repetition of jobs, decoded left to right). Following the notation of Díaz et al. (2020) and Díaz et al. (2023a), let $\text{PM}_o(\pi)$ and PJ_o denote the predecessors of task o in its machine (according to π) and in its job. The *semi-active* schedule (Sprecher et al., 1995) starts each task at

$$\mathbf{s}_o(\pi) = \max(\mathbf{c}_{\text{PJ}_o}(\pi), \mathbf{c}_{\text{PM}_o(\pi)}(\pi)), \quad \mathbf{c}_o(\pi) = \mathbf{s}_o(\pi) + \mathbf{p}_o, \quad (3)$$

so starting and completion times of all tasks are themselves intervals. The problem can be stated as

$$\min_R \mathbf{C}_{\max} \quad (4)$$

$$\text{s.t. } \mathbf{C}_{\max} = \max_{1 \leq j \leq n} \{\mathbf{c}_{o_{jm}}\}, \quad (5)$$

$$\underline{s}_{o_{jl}} \geq \underline{c}_{o_{j,l-1}}, \quad \overline{s}_{o_{jl}} \geq \overline{c}_{o_{j,l-1}}, \quad 2 \leq l \leq m, \quad 1 \leq j \leq n, \quad (6)$$

$$\underline{s}_o \geq \underline{c}_{o'} \vee \underline{s}_{o'} \geq \underline{c}_o, \quad \overline{s}_o \geq \overline{c}_{o'} \vee \overline{s}_{o'} \geq \overline{c}_o, \quad \forall o \neq o' : \mu(o) = \mu(o'), \quad (7)$$

where Eq. (5) defines the interval makespan as the component-wise maximum of the job completion times, Eq. (6) enforces precedence within each job and Eq. (7) forbids overlaps on each machine, in both interval components (Díaz et al., 2020, 2023a). The minimum in (4) is taken with respect

to an interval ranking R , since intervals admit no natural total order and any ranking embodies a modeling choice (Ishibuchi and Tanaka, 1990). We rank schedules lexicographically, comparing upper bounds first and using the lower bound only to break ties:

$$\mathbf{a} \preceq \mathbf{b} \iff \bar{a} < \bar{b} \vee (\bar{a} = \bar{b} \wedge \underline{a} \leq \underline{b}), \quad (8)$$

one of the standard rankings of the IJSP literature and the one found by Díaz et al. (2020) to yield the most robust schedules. All methods in this paper, the GP rules and every baseline, share one implementation of this decoder and evaluator, so reported differences cannot stem from evaluator discrepancies.

Performance is reported as the relative error of the expected makespan with respect to a crisp reference bound LB of each instance,

$$\text{RE} = \frac{E[\mathbf{C}_{\max}] - \text{LB}}{\text{LB}} \times 100, \quad E[\mathbf{C}_{\max}] = \frac{\underline{C}_{\max} + \overline{C}_{\max}}{2}, \quad (9)$$

where $E[\mathbf{C}_{\max}]$ is the midpoint of the interval makespan, which the IJSP literature calls its expected makespan because it is the expectation under a uniform distribution on $[\underline{C}_{\max}, \overline{C}_{\max}]$; it serves here only as a point summary of the interval, and no distributional assumption enters the problem. LB is a bound of the underlying deterministic instance: Taillard’s bounds (Taillard, 1993) for the Taillard-derived instances, the best known ones (Díaz et al., 2023b) for the classical set. It serves as an indicative common reference, not as a bound for the interval problem, for which none is available.

A schedule is eventually *executed* under one realization of the durations, and a second quantity of interest is how well its a-priori interval predicts that execution. Measuring the gap between a predicted schedule and its execution is the classical notion of schedule robustness (Leon et al., 1994),



Figure 1: An individual is a priority expression tree; leaves are interval-aware attributes of the candidate operation (here the illustrative rule $SLACK/PT + EST - WKR \cdot PTW$).

here specialized to interval predictions. Following Díaz et al. (2020), the $\bar{\varepsilon}$ robustness of a schedule with a fixed processing order is the average relative deviation of its executed makespan from the predicted expected value, over K realizations of the durations:

$$\bar{\varepsilon} = \frac{1}{K} \sum_{k=1}^K \frac{|C_{\max}^{\text{ex},k} - E[\mathbf{C}_{\max}]|}{E[\mathbf{C}_{\max}]}, \quad (10)$$

where $C_{\max}^{\text{ex},k}$ is the crisp makespan obtained by executing that same order under the durations of realization k . A low $\bar{\varepsilon}$ therefore means that the executed makespan stays close to its a-priori expectation.

4. GP Hyper-Heuristic for the IJSP

We describe the method from two complementary viewpoints: as a *hyper-heuristic*, that is, what an individual is and how it turns an IJSP instance into a schedule (Section 4.1); and as an *evolutionary algorithm*, that is, how the population of rules is initialized, varied and selected (Section 4.2).

4.1. Hyper-heuristic view: from expression trees to schedules

An individual is a priority expression tree (Figure 1). Leaves are drawn from the interval-aware terminal set of Table 1, and internal nodes from the

Table 1: Interval-aware terminal set, with exact definitions. For an eligible operation o of job j on machine $\mu(o)$: $\mathbf{p}_o = [\underline{p}_o, \bar{p}_o]$ is its duration interval; $\mathbf{C}_j$ and $\mathbf{C}_{\mu(o)}$ are the interval completion times of the last scheduled operation of job j and of machine $\mu(o)$; $\mathcal{R}_o$ is the set of operations of j after o ; $\mathcal{E}$ is the current eligible set. Terminals are evaluated on *raw* (unnormalized) values.

Terminal	Definition	Role
PT	$\bar{p}_o$	worst-case processing time
PTW	$\bar{p}_o - \underline{p}_o$	duration uncertainty
EST	$\bar{e}_o = \max(\bar{C}_j, \bar{C}_{\mu(o)})$	worst-case earliest start
ESTW	$\bar{e}_o - \underline{e}_o$, with $\underline{e}_o = \max(\underline{C}_j, \underline{C}_{\mu(o)})$	start uncertainty
WKR	$\sum_{q \in \mathcal{R}_o} \bar{p}_q$	worst-case work remaining
WKRW	$\sum_{q \in \mathcal{R}_o} (\bar{p}_q - \underline{p}_q)$	remaining-work uncertainty
NOR	$ \mathcal{R}_o $	operations remaining
SLACK	$\bar{e}_o - \min_{o' \in \mathcal{E}} \bar{e}_{o'}$	start slack within eligible set
ONE	1	constant

function set

$$\mathcal{F} = \{ +, -, \times, \div_p, \min, \max, \text{neg} \}, \quad (11)$$

where neg is the unary sign change and $\div_p$ denotes protected division, defined as $a \div_p b = a/b$ if $|b| > 10^{-9}$ and a otherwise. The rule is deployed inside a semi-active schedule generation scheme (SGS): starting from the empty schedule, at each of the nm decision points the set $\mathcal{E}$ of eligible operations (those whose job predecessor is already scheduled) is formed, the tree is evaluated on the attributes of every $o \in \mathcal{E}$, and the operation with *lowest* priority value is appended at its earliest feasible start on its machine, Eq. (3).

Because the lowest value is dispatched, a criterion that seeks to *maximize* an attribute must be expressed through the unary neg operator. Classic

rules thus have counterparts in this search space: the single-node tree PT is shortest processing time on the upper endpoints, and $\text{neg}(WKR)$ is most work remaining. They differ from the SPT and MWKR baselines of Section 5.2 only on ties, which the trees break by job index and the baselines by the lower endpoint, Eq. (8). We inject both trees as seeded rules in the initial population.

Priority ties at dispatch time are broken by eligible set order (lowest job index first), which is deterministic; hence a rule maps each instance to exactly one schedule.

4.2. Evolutionary components

The initial population is built with the *ramped half-and-half* method (Koza, 1992; Poli et al., 2008): each individual draws its maximum depth uniformly from $\{2, 3, 4\}$ and is generated, with equal probability, by the *full* method, which expands every branch with function nodes down to that depth, or by the *grow* method, which may close any branch earlier by placing a terminal (probability 0.3 per node in our implementation). In both cases internal nodes are drawn uniformly from $\mathcal{F}$ and leaves uniformly from the terminal set. The last two members of the population are not generated at random: they are the two seeded rules of Section 4.1, inserted directly. Every individual is evaluated by the mean RE of its deterministic constructive pass over the four training instances.

Each generation then proceeds as follows. Parents are chosen by tournament, drawing k individuals uniformly without replacement from the current population and returning the fittest of them. With probability p_c two parents are recombined by *subtree crossover* (Koza, 1992; Poli et al., 2008): a node is drawn uniformly at random in each parent, leaves included, and the

Algorithm 1 GP hyper-heuristic for the IJSP

```
1:  $P \leftarrow \text{ramped half-and-half}(\text{pop} - 2) \cup \{\text{SPT}, \text{MWKR}\}$ 
2:  $\text{evaluate}(P)$   $\triangleright$  mean RE of one constructive pass on the training
   instances
3: for  $g = 1$  to  $\text{gens}$  do
4:    $P' \leftarrow \text{elite}(P, e)$   $\triangleright$  elitism
5:   while  $|P'| < \text{pop}$  do
6:     if  $\text{rand}() < p_c$  then
7:        $p_1, p_2 \leftarrow \text{tournament}(P, k), \text{tournament}(P, k)$ 
8:        $\text{child} \leftarrow \text{subtreeCrossover}(p_1, p_2)$ 
9:     else
10:       $\text{child} \leftarrow \text{subtreeMutation}(\text{tournament}(P, k))$ 
11:    end if
12:     $P' \leftarrow P' \cup \{\text{child}\}$   $\triangleright$  fitness  $\infty$  if size  $> c$ 
13:  end while
14:   $P \leftarrow \text{evaluate}(P')$ 
15: end for
16: return best rule in  $P$ 
```

offspring is a copy of the first parent with the subtree rooted at its chosen node replaced by the one rooted at the second parent's. Unlike the classical two-offspring formulation, we keep a single offspring per recombination. With the complementary probability $1 - p_c$ a single parent undergoes *subtree mutation* (Koza, 1992; Poli et al., 2008): a node is again drawn uniformly at random, and the subtree rooted at it is discarded and replaced by a freshly generated random tree, built with the grow method and maximum depth 3 over the full primitive set. Offspring whose size exceeds a cap of c nodes

Table 2: Evolution parameters: the configuration selected by irace (Section 5.3), used for every experiment reported in this paper unless a section states otherwise.

Population / generations	100 / 50
Initialization	ramped half-and-half (depths 2–4)
Seeded individuals	PT and $\text{neg}(WKR)$
Selection	tournament, size $k = 7$
Crossover / mutation prob.	$p_c = 0.77$ / 0.23 (subtree, depth ≤ 3)
Elitism	$e = 2$
Tree-size cap	$c = 30$ nodes (violations: fitness ∞)
Fitness	mean RE, deterministic rollout, TA11–TA14
Seeds	1–30 (RNG of evolution and environment)

receive infinite fitness and are thereby excluded from selection. The depth bounds above apply only to trees created from scratch, at initialization and inside mutation; during the run only the node count is bounded, so recombination may produce deeper expressions.

Replacement is fully generational, the offspring becoming the next population, except for the e best individuals of the current generation, which are copied into it unchanged. The process stops after a fixed number of generations and returns the best rule found. Algorithm 1 summarizes the generational loop and Table 2 lists the parameter values.

4.3. Randomized multi-start extension

A deterministic rule produces one schedule. For matched comparisons with sampling-based methods we wrap the evolved rule in GRASP-style randomization (Feo and Resende, 1995): with probability 0.1 the dispatched operation is drawn uniformly from the eligible set, otherwise the rule decides; sample 0 remains deterministic. The wrapper, which we call GP-best-

of- N , reports the lexicographically best of its N schedules. This uniform perturbation of a single rule is the simplest member of a family of randomized rule-based builders; an alternative swaps among a *set* of evolved rules during construction (Gil-Gala et al., 2025a). GP-best-of- N is a device for comparison rather than a deployment mode: it places the evolved rule on the same footing as sampling-based methods when methods are compared at a given number of schedule constructions and, as with GRASP and other multi-start constructive schemes, what improves with N is the dispersion of the sampled schedules rather than any individual decision. Its cost is reported in Sections 6.4 and 6.5.

5. Experimental Setup

5.1. Benchmark instances and data split

We use two benchmarks, both taken from the IJSP literature (Díaz et al., 2020, 2023b,a): the 70 interval versions of Taillard’s instances (Taillard, 1993) (TA1–TA70, seven size classes from 15×15 to 50×20), and the 12 classical instances on which the published IJSP metaheuristics report results (FT10, FT20 (Fisher and Thompson, 1963), seven instances of the La family (Lawrence, 1984) and ABZ7–9 (Adams et al., 1988), from 10 to 20 jobs and 5 to 15 machines). Both were obtained from their crisp counterparts with the generation scheme introduced by Díaz et al. (2020): for every operation a value δ is drawn uniformly at random in $[0, 0.15p]$, where p is its original duration, and that duration becomes the integer interval $[p-\delta, p+\delta]$, machine routes being unchanged. The interval is symmetric around p , so the crisp instance is exactly the midpoint scenario of its interval version. Both sets are released with this paper (see Data availability).

The 70 interval Taillard instances are split three ways. Rules are evolved on TA11–TA14 (20×15). TA15–TA20 form the validation set: they never contribute to fitness, but the design and configuration decisions of this paper, including the irace study of Section 5.3, were selected on their performance. The remaining 60 instances are unseen by every design and configuration decision and, together with the 12 classical instances, to which the rules are applied without any re-evolution (Section 6.3), constitute the generalization test.

Thirty independent evolutions, differing only in their random seed, yield thirty rules. The one with the lowest mean RE on the validation set is the *featured rule*: the 60 held-out Taillard instances and the 12 classical ones play no part in this or in any other choice of the paper. Results are reported as the mean over the 30 rules, which no selection touches, beside the featured rule’s value, except where a section states that a single rule is examined.

For the analysis of Section 7.7 the 70 Taillard instances were also regenerated with intervals skewed to the right. The crisp duration is recovered from the midpoint of the symmetric version, which the scheme above preserves exactly, and the bounds are drawn as $p - \text{round}(pU[0, 0.05])$ and $p + \text{round}(pU[0, 0.25])$, so that a duration may fall slightly below its nominal value and considerably above it. Machine routes are unchanged and the crisp Taillard bounds still apply. Over the 37,250 operations the mean total width is 14.3% of p against 12.9% for the symmetric scheme: the two benchmarks differ above all in the shape of the intervals, and the asymmetric intervals are also wider on average, by 1.4 points of p .

5.2. Reference methods

The constructive baselines are the dispatching rules of Section 2 (SPT, LPT, EST, MWKR, MOR and CR) and the two Giffler–Thompson combinations, G&T-SPT and G&T-MWKR, each applied in one deterministic pass. Each ranks candidates by its own criterion under the lexicographic order of Eq. (8); Section 7.3 checks that this choice does not handicap them. A dispatcher that chooses uniformly among the eligible operations serves as a reference point.

The sampled variant of the evolved rule is GP-best-of- N (Section 4.3). The published IJSP metaheuristics are implemented in C++ and report results under their own stopping rules, which does not allow them to be compared with the evolved rule at a matched budget. For that comparison we implemented our own version of the genetic algorithm of Díaz et al. (2020) in Python, on the decoder and evaluator of Section 3, so that it and the evolved rule differ only in the method. A solution is a permutation with repetition of jobs, decoded by the semi-active scheme of Eq. (3) and compared by the ranking of Eq. (8). A population of 250 individuals is initialized at random; parents are chosen by tournaments of size 3 and recombined with probability 0.9 by a job-based order crossover, in which a random subset of jobs keeps its positions from the first parent and the remaining positions are filled in the order they take in the second; offspring are mutated with probability 0.2 by swapping two positions, and the two best individuals pass unchanged to the next generation. The population size is that of the published algorithm, and of four configurations tried at 2×10^5 constructions on the training instances it was the best. The algorithm is stopped after a given time per instance, the budget measure of Section 6.5, and records its incumbent solution each time the number of schedule constructions grows

Table 3: Hyperparameter space explored by irace (200 experiments; cost = validation-set RE of the evolved rule) and the winning configuration.

Parameter	Range / values	irace winner
Tournament size	{2, 3, 4, 5, 7}	7
Crossover prob. p_c	[0.5, 0.95]	0.77
Tree-size cap	{20, 30, 40, 60}	30
Elitism	{1, 2, 4}	2

by a factor of $2^{1/4}$.

5.3. Hyperparameter configuration with irace

Four parameters of the evolution were configured with irace (López-Ibáñez et al., 2016): tournament size, crossover probability, tree-size cap and elitism. Population size and number of generations were held fixed, so that every configuration is evaluated under the same evolution budget. The budget was 200 experiments, each one a full evolution on the training instances, scored by the resulting rule’s RE on the validation set. Table 3 lists the parameter space and the winning configuration (Table 2). Of the four elite configurations irace retains, three favor a high selection pressure (tournament size 7) and all four fall in a narrow crossover band, 0.72–0.77; the tree-size cap does not discriminate, the elites spanning caps of 20, 30 and 40.

5.4. Evaluation measures and statistical analysis

Quality is reported as the relative error RE of Section 3. The spread of a prediction is the relative width of the interval makespan, $W = (\overline{C}_{\max} - \underline{C}_{\max})/E[\mathbf{C}_{\max}] \times 100$, and executional robustness is measured by $\bar{\varepsilon}$ of Eq. (10) and by the absolute deviation $|\Delta|$, the same average

taken before dividing by $E[\mathbf{C}_{\max}]$ and expressed in time units. Both are averages over realizations; the tail of the same realizations is described by the conditional value-at-risk at level $\alpha = 0.95$, $\text{CVaR}_{0.95}$, the mean of the 5% largest values of a quantity, which is applied to the overrun $C_{\max}^{\text{ex}} - E[\mathbf{C}_{\max}]$ of the executed makespan beyond its prediction, in time units, and to the executed makespan itself, expressed as RE over the same bound LB. All of these are estimated by Monte Carlo over realizations of the durations, as described in Section 7.6. Lower is better on every measure.

Two kinds of comparison appear, with different experimental units. Methods are compared instance by instance, pairing their values on the same instances. Arms of evolutions, which differ in terminal set, fitness or benchmark, are compared seed by seed, pairing the rules evolved with the same seed after averaging each rule over the instances. In both cases the test is the two-sided Wilcoxon signed-rank test with exact p -values, reported with the normal-approximation statistic z and, as effect size, the matched-pairs rank-biserial correlation $|r|$. Holm’s correction is applied within each family of related comparisons, and where it changes a verdict the text says so.

5.5. Computing environment

All experiments ran on a desktop computer with an AMD Ryzen 5 3400G processor (4 cores, 8 threads) and 32 GB of RAM under Windows 10 Pro. The hyper-heuristic, the decoder, the baselines and the genetic algorithm are implemented in Python 3.12 with NumPy 2.3, statistical tests use SciPy 1.17, and the configuration study used irace 4.4.3 (López-Ibáñez et al., 2016). All times reported are wall-clock times of a single simulator, an implementation

of the decoder of Section 3 in the same language that builds exactly the same schedules as the one used during evolution, measured with six runs executing in parallel; they are comparable with one another, not with times obtained on other hardware or in other languages.

6. Results

6.1. Evolution behavior and seed stability

Over 30 independent evolutions, training RE converges to 14.9 ± 1.5 (range 12.4–19.7), with most of the improvement realized by generation 25–30 (Figure 2a). The same panel evaluates the best rule of each generation on the instances the fitness never sees, and there the improvement is concentrated in the first ten generations. From generation 10 to 50 the median training RE falls from 17.09 to 14.57, while that on the 60 test instances falls only from 19.32 to 18.71 and that on the validation set hardly moves, from 19.85 to 19.69: the later generations specialize the rule to the four training instances more than they improve it elsewhere. Over the same generations the best rules grow to a median of 27 nodes, just below the cap of 30 (Figure 2b), and interval-width terminals make up about a fifth of their leaves throughout (Figure 2c). Figure 3 shows the featured rule, obtained with seed 1: 26 nodes over 12 leaves. Every terminal of the tree is non-negative by construction and processing times are at least one time unit, so both guards it contains resolve to the same branch at every decision. On the problem domain the tree is therefore exactly equivalent to the expression

$$SLACK^2 + 2PT - WKR - WKRW - 1. \quad (12)$$

We verified this identity at each of the 37,250 dispatching decisions the rule takes on the benchmark (1.29 million candidate evaluations).



Figure 2: The best rule of each generation over the 30 independent evolutions: (a) its RE on the training, validation and test instances, (b) its size in nodes, and (c) the share of interval-width terminals among its leaves. Lines are medians, means in (c); shaded bands are interquartile ranges.

6.2. Position among constructive methods

Table 4 reports every constructive baseline under the protocol of this paper, one deterministic pass per instance, together with the random dispatcher of Section 5.2. Since these rules are deterministic, the standard deviation is taken across instances and measures how uniform a method is over the benchmark rather than any run-to-run variability; the random dispatcher, the one stochastic entry, is averaged over 10 dispatches per instance. Times are averages over the 70 instances with 5 repetitions each. The GP entries are the mean over the 30 evolved rules, the mean being what a single evolution should be expected to yield, and the best of them, both timed as evolved, and the best rule in its simplified form, which takes the same decisions.

Two observations are needed to read the table correctly. First, the plain rules that ignore machine availability (SPT, LPT and the critical ratio CR, the latter being a due-date rule applied here without due dates) perform



Figure 3: The featured rule (seed 1), which attains 17.71% RE over the 70 instances. Circles are functions and shaded boxes the interval-aware terminals of Table 1. The tree is shown exactly as evolved; Eq. (12) gives its simplified form.

worse than random in this setting. This is a property of the schedule generation scheme, not a defect of the criteria: because a solution is decoded as a permutation whose operations are appended to their machines, systematically dispatching by processing time alone produces highly unbalanced permutations. The same criterion embedded in the Giffler–Thompson conflict set, which restores classical dispatching semantics, behaves as expected (SPT: 627% as a plain rule versus 70.6% under G&T). Second, the criteria that do account for the state of the shop (EST, MOR, MWKR) are better than random, and the strongest baseline overall is G&T-MWKR at 29.5%, which is the reference we use throughout.

Against that strongest baseline, the evolved rules average 18.99 ± 1.33

Table 4: Constructive baselines on the 70 interval Taillard instances: mean RE (%), standard deviation across instances, and mean time in milliseconds of one constructive pass.

Method	RE (%)	sd	Time (ms)
LPT	703.6	168.0	21.3
SPT	627.2	152.2	20.9
CR	625.7	140.6	22.5
<i>Random dispatcher</i>	<i>127.2</i>	<i>13.9</i>	<i>2.5</i>
G&T-SPT	70.6	10.0	29.5
MWKR	64.7	10.4	28.7
MOR	45.5	10.1	26.7
EST	42.3	10.6	22.3
G&T-MWKR	29.5	6.4	37.8
GP rule (mean of 30)	18.99	4.96	47.7
GP rule (best of 30)	17.71	5.23	44.7
simplified, Eq. (12)	17.71	5.23	32.6

RE over the 30 evolutions, and the best rule attains 17.71%.

The gap holds at the level of individual instances: the best evolved rule outperforms each of the eight deterministic baselines on every one of the 70 instances, and a paired Wilcoxon signed-rank test rejects equality against each of them at $p < 10^{-6}$ ($z = -7.3$); the conclusion is unchanged under Holm’s correction for the eight comparisons. The narrowest margin over the whole comparison is 3.9 points, against G&T-MWKR on a single 20×15 instance; against the best plain rule (EST) the smallest margin is 6.3 points. Per-instance figures are given in Supplementary Table S3. This quality comes at a small evaluation cost. A pass of the best rule takes

44.7 ms as evolved, against 26.7 ms for MOR and 37.8 ms for G&T-MWKR: evaluating a tree of up to 30 nodes on every eligible operation costs more than reading one attribute, but not by an order of magnitude, and in its simplified form the same rule takes 32.6 ms. Both cost and quality thus place the evolved rule with the constructive methods, while closing much of the quality gap that separated them from search-based methods; that comparison is the subject of Sections 6.4 and 6.5.

6.3. Generalization across sizes and instance families

Table 5 reports single-rollout RE per size class over all 70 instances, as mean $\pm$ standard deviation across the 30 evolved rules and for the single best rule. Evolved only on 20×15 , the best rule attains its lowest RE on the unseen 50×15 (13.8%), and the same pattern holds on average (13.7 ± 1.5). Because all terminals are per-operation attributes with no size-bound quantities, transfer requires no retraining or rescaling.

The ranking of the classes reflects the difficulty of their instances more than their distance from the training size. In the two 50-job classes, where the rule does best, every crisp counterpart has been solved to optimality (Coupvent des Graviers et al., 2025) and the bounds used here equal the optimal makespans, so on them RE is measured against the optimum. The error is largest on 30×20 , the class with the most open instances, nine of ten. The reference bound of an open instance lies below its unknown optimum, so part of the error measured there belongs to the bound: the best known solutions of that class exceed the bounds used here by 2.7% on average.

The rules also transfer to the second benchmark, whose 12 classical instances differ from the Taillard ones in origin, size and structure. Table 6

Table 5: RE (%) per size class, single deterministic rollout, over all 70 Taillard interval instances: mean $\pm$ sd across the 30 independently evolved rules, and the best evolved rule.

	15 $\times$ 15	20 $\times$ 15	20 $\times$ 20	30 $\times$ 15	30 $\times$ 20	50 $\times$ 15	50 $\times$ 20	All
Mean of 30	18.1 $\pm$ 1.3	18.0 $\pm$ 1.5	20.9 $\pm$ 1.7	21.4 $\pm$ 1.7	25.7 $\pm$ 1.9	13.7 $\pm$ 1.5	15.1 $\pm$ 1.5	18.99 $\pm$ 1.33
Best rule	15.7	16.6	18.1	21.1	24.1	13.8	14.6	17.71

gives the featured rule’s single-pass RE on each, together with GP-best-of- N at $N = 64$ and $N = 1024$ and the published results of the IJSP metaheuristics, which Section 6.4 discusses.

Transfer is a property of the method rather than of one rule: over the 30 evolved rules the deterministic pass averages $19.04 \pm 1.97\%$ RE on these instances, single rules ranging from 15.3% to 23.3%. Since the two benchmarks use different reference bounds (Section 3), RE values do not compare across them, but margins over baselines evaluated on the same instances do, and that margin is preserved: 2.40 times over MOR and 1.58 over G&T-MWKR on the classical set, against 2.40 and 1.55 on the benchmark the rules were evolved for.

6.4. Comparison with published metaheuristics

On the 12 classical instances the evolved rules can be compared with the published results of the IJSP metaheuristics on their own benchmark and metric (Table 6). Those methods belong to different algorithmic families, evolutionary computation and swarm intelligence: a genetic algorithm evolving a population of permutations (Díaz et al., 2020) and three artificial bee colony variants (Díaz et al., 2023b,a), each of which runs a separate search on every instance.

The evolved rules are not competitive with the per-instance metaheuristics: the purpose-built ABC variants remain clearly ahead (5.5–6.4%), as

Table 6: RE (%) on the 12 classical interval instances: our methods (deterministic single pass and GP-best-of- N) versus the published average results (30 runs per instance) of the IJSP metaheuristics GA (Díaz et al., 2020), ABC_{E3} , fEABC (Díaz et al., 2023b) and ESABC (Díaz et al., 2023a). LB is the best-known lower bound of the expected makespan.

Inst.	LB	Ours (zero-shot)			Published (avg of 30)			
		GP	GP-64	GP-1024	GA	ABC_{E3}	fEABC	ESABC
FT10	930	20.4	8.4	6.6	5.2	4.1	3.5	3.0
FT20	1165	13.2	12.1	10.8	4.4	1.7	1.8	1.8
La21	1046	15.0	14.4	9.0	5.0	5.0	4.2	4.0
La24	935	10.1	8.8	8.6	6.3	5.1	4.9	5.0
La25	977	11.8	11.6	7.6	5.1	3.9	3.4	2.7
La27	1235	25.0	13.7	11.3	10.2	4.7	4.6	4.1
La29	1152	13.4	14.2	11.0	14.2	8.6	7.4	7.0
La38	1196	15.0	13.3	9.4	9.2	6.9	6.1	5.8
La40	1222	13.3	12.8	9.9	8.7	4.2	4.6	4.1
ABZ7	656	19.7	14.3	11.7	12.5	7.3	6.6	6.7
ABZ8	645	24.7	22.1	17.7	18.5	12.1	11.1	10.9
ABZ9	661	33.4	25.3	22.1	18.0	13.1	11.6	11.2
Mean		17.9	14.2	11.3	9.8	6.4	5.8	5.5

expected of iterative search, and even wrapped as GP-1024 the featured rule reaches 11.3% against the genetic algorithm’s published 9.8%, better on 3 of the 12 instances (paired Wilcoxon $z = 1.96$, $p \approx 0.05$). What the comparison locates is the constructive frontier: in a single pass the evolved rules average 19.04% on these instances, against 30.1% (G&T-MWKR) and 45.8% (MOR) for the deterministic baselines, and the best rule with a 1024-sample budget comes within 1.5 points of a population-based method that searches every instance separately.

The two families also differ in how they spend computation: GP-1024 evaluates 1024 mutually independent samples per instance, whereas the metaheuristics evolve a population of 250 individuals (Díaz et al., 2020, 2023b) through successive generations, each of which depends on the previous one. In wall-clock terms, a single pass of the rule, in its simplified form, over these instances takes 2–11 ms in our Python implementation, against the 0.5–2.2 s reported for the genetic algorithm and the 1.9–6.8 s reported for fEABC on the same instances (Díaz et al., 2023b), both implemented in C++. GP-1024, however, requires 2.1–10.4 s depending on instance size, and is therefore *not* faster than the published genetic algorithm, and of the same order as fEABC.

6.5. Comparison at matched budget with a reimplemented genetic algorithm

The comparisons of Section 6.4 set our Python implementation against published C++ results, each obtained under its authors’ own stopping rule. For a comparison at matched budget we use our implementation of the genetic algorithm of Díaz et al. (2020) (Section 5.2). It is a faithful reference: on the 12 classical instances it reaches 8.3% at 2^{19} schedule constructions, and the 9.8% published for the original after about 13 s per instance, and it levels off short of the 5.5% of ESABC, a distance due to the local search of the ABC variants rather than to budget. The rule and the genetic algorithm share decoder, evaluator and language, and both are implemented without avoidable overhead: the rule is compiled into a single function that computes only the terminals it reads, and the decoder and crossover of the genetic algorithm are streamlined in the same way, both producing exactly the schedules of a direct implementation. Budget is measured in seconds per instance, which charges each method for what a schedule costs it: a sample

of the rule evaluates the tree on every eligible operation at every step, and costs as much as 15 to 43 decodings of the genetic algorithm, more the larger the instance. The comparison is run on the 12 classical instances, with 30 runs of 900 s per instance and method, and on three Taillard classes that differ only in the number of jobs, 15×15 , 30×15 and 50×15 , with 3 runs per instance and horizons of 900, 1800 and 3600 s, chosen to extend well beyond the point where the curves cross (Figure 4).

The four instance sets order the methods in the same way. With little time the sampled rule is the best use of it: after one second per instance on the classical instances it stands at 11.6%, against 14.3% for the seeded genetic algorithm and 27.1% for the unseeded one. The genetic algorithm matches a single pass of the rule after 2 to 46 s per instance, and overtakes the sampled rule later, the later the larger the instance: from about 10 s on the classical instances, 25 s on 15×15 , 83 s on 30×15 and 300 s on 50×15 . From there on it stays below, and at the end of the horizon both genetic algorithms are ahead of the sampled rule in mean. The difference is significant for the seeded version on the classical instances (better on 11 of 12, $p = 0.012$) and on 50×15 (9 of 10, $p = 0.006$), where the unseeded version is also significantly better ($p = 0.020$), and not on the two smaller Taillard classes. The advantage of the rule is therefore not that it optimizes better, but that it is effective at budgets too small for search to be so, and how small depends on the size of the instance.

Seeded with the rule’s permutation, the genetic algorithm starts from the rule’s solution instead of from random permutations, and for a while it is far ahead of its unseeded version: significantly, on every instance set, after one second per instance, and for up to 30 s on 50×15 . It also overtakes the sampled rule before the unseeded version on three of the four sets, at about

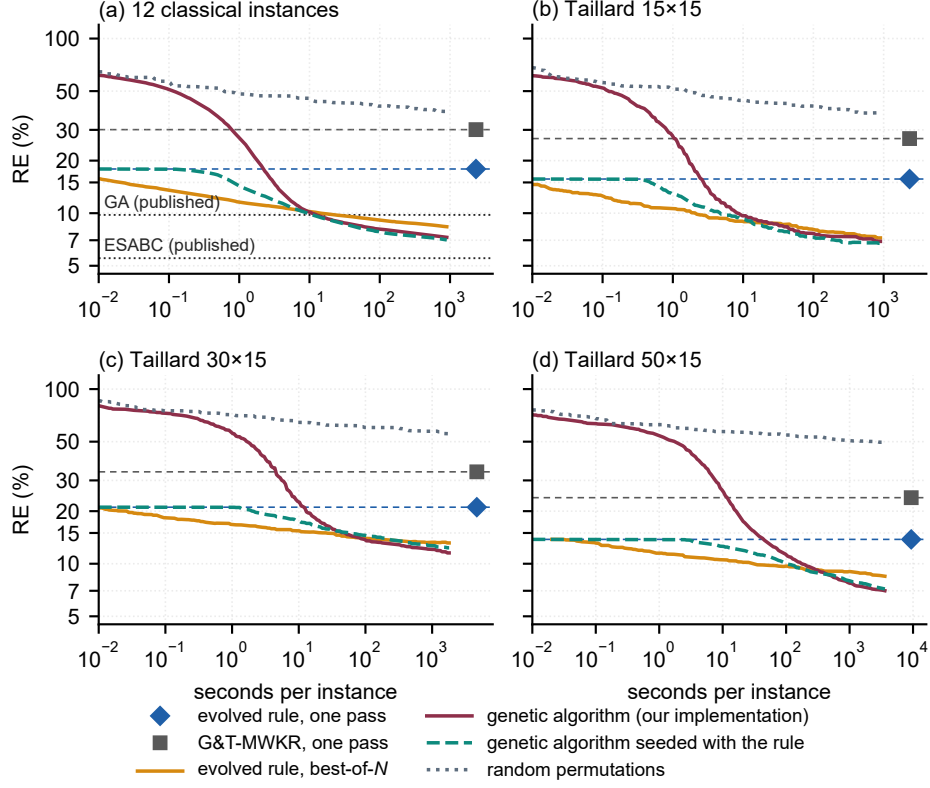


Figure 4: RE against seconds per instance on the 12 classical instances (a) and on three Taillard classes with 15 machines, 15x15 (b), 30x15 (c) and 50x15 (d). Each curve is the mean over the instances of the mean over runs of the RE of the incumbent solution, with 30 runs per instance in (a) and 3 in (b)–(d). The single-pass methods are horizontal reference lines, marked at their right end; in (a) the dotted lines are the results published for the genetic algorithm and ESABC.

7 s on the classical instances, 14 s on 15x15 and 170 s on 50x15; on 30x15 it follows the sampled rule closely until about 280 s. Given enough time, however, the search reaches the same quality without the seed: from 100 s per instance on, the two versions are not distinguishable on any set, and over the 42 instances at the end of their horizons the seeded version is better on

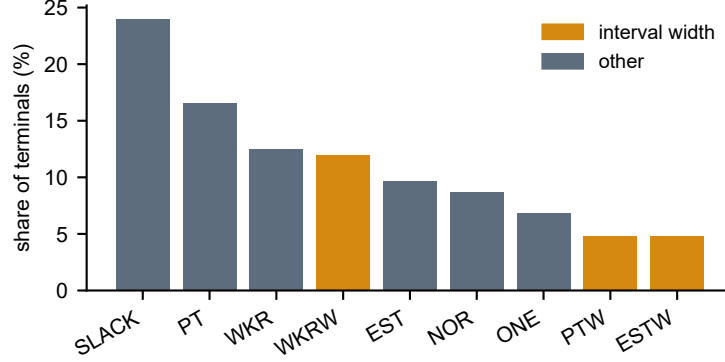


Figure 5: Terminal usage across the 30 evolved rules, as a share of all terminals. Interval-width terminals (amber) expose the magnitude of the uncertainty to the rule.

20 ($p = 0.74$; Supplementary Table S4 gives the tests over time). An evolved dispatching rule is thus the cheapest way to give a per-instance metaheuristic a good first incumbent, the role constructive heuristics commonly play in the initialization of population methods (Kazimipour et al., 2014), and its value as a seed lies in short budgets.

7. Analysis

7.1. Rule anatomy and interpretability

The output of GP is a readable expression. Evolved rules are compact, with a mean tree size of 26 nodes (20–30) and mean depth 9, so a rule can be inspected and simplified by hand. Figure 5 reports how often each terminal is used across the 30 evolved rules. The most frequent are the classical scheduling attributes, slack (*SLACK*), work remaining (*WKR*) and earliest start (*EST*), together with worst-case processing time (*PT*), indicating that the evolution rediscovers known dispatching logic rather than exploiting benchmark artifacts. The *interval-width* terminals (*PTW*, *ESTW*, *WKRW*)



Figure 6: Sensitivity of the featured rule to the two weights of its simplified form, Eq. (12): mean RE over the 70 instances when each weight is varied and the other is held at its evolved value. Dashed lines mark the evolved weights, the dotted line the mean over the 30 evolved rules. $\beta = 0$ is the rule with its interval-width term switched off.

account for 21.7% of all terminal usage and appear in 27 of the 30 evolved rules. Whether that width information translates into better schedules is quantified in Sections 7.4–7.6.

Since the lowest value is dispatched, the featured rule, Eq. (12), reads directly: $-WKR$ is most-work-remaining, $2PT$ is shortest-processing-time at double weight, $SLACK^2$ penalizes quadratically any operation that cannot start at the earliest available time, and the constant is immaterial to the ranking. The remaining term, $-WKRW$, is the interval-aware one: among jobs with comparable remaining work the rule advances those whose remaining work is *more uncertain*, resolving that uncertainty early rather than carrying it to the end of the schedule. The evolved rule is thus a weighted blend of MWKR and SPT, regularized by a quadratic earliest-start term and tilted by the uncertainty of the work ahead.

The simplified form of the featured rule, Eq. (12), also admits a direct

sensitivity analysis: it weighs PT by $\alpha = 2$ and $WKRW$ by $\beta = 1$, and we swept each weight over the 70 instances holding the other at its evolved value (Figure 6). Neither curve is unimodal: over the sampled grid α has five local minima and β three, and since each point is a deterministic evaluation over the 70 instances, that ruggedness belongs to the landscape rather than to sampling noise. The evolved $\alpha = 2$ is nonetheless the global minimum of its sweep; the evolved β is not, giving 17.71% against the 17.54% of $\beta = 0.5$. A post-evolution weight sweep is thus a cheap refinement for any simplified rule. The multimodality becomes relevant if the sweep is replaced by local descent: started at $\alpha = 4$, a descent converges to a secondary minimum 0.6 points above the global one. Setting $\beta = 0$ removes the width term from the deployed rule and costs 0.5 points of RE (18.23 against 17.71).

7.2. A worked example

A 3×3 instance makes the tilt introduced by $-WKRW$ visible by comparing the rule with its variant without the width term, $\beta = 0$ in the notation of Section 7.1. The instance has three jobs of three operations: J_1 visits M_3, M_2, M_1 with durations $[28, 30]$, $[20, 26]$, $[24, 32]$; J_2 visits M_1, M_3, M_2 with $[26, 30]$, $[10, 12]$, $[16, 18]$; and J_3 visits M_3, M_1, M_2 with $[22, 26]$, $[14, 14]$, $[27, 31]$.

The two traces coincide in the first two decisions and diverge at the third, where M_3 has just been released by o_{11} and the eligible set is $\{o_{12}, o_{22}, o_{31}\}$. Table 7 gives the terminals and the resulting scores. All three candidates can start at the same time, so SLACK vanishes and the decision rests on the remaining terms. Without the width term the ranking puts o_{22} (5) ahead of o_{31} (6): the short operation of the job with little work remaining ranks first because of the $2PT$ term. The width term contributes $-WKRW$, which is

Table 7: Terminals and priority scores at the third dispatching decision of the worked example.

Eligible	Duration	WKR	WKRW	SLACK	Eq. (12)	$\beta = 0$
o_{12}	[20, 26]	32	8	0	11	19
o_{22}	[10, 12]	18	2	0	3	<u>5</u>
o_{31}	[22, 26]	45	4	0	<u>2</u>	6

−4 for o_{31} against −2 for o_{22} , and that reverses a one-point gap. The rule therefore dispatches o_{31} , whose job has more uncertain remaining work than that of o_{22} , [41, 45] against [16, 18]. In this same state, SPT would dispatch o_{22} and MWKR o_{31} .

Figure 7 shows what follows. Dispatching J_3 early schedules its remaining operations while the machines are still available, and the makespan is $\mathbf{C}_{\max} = [103, 117]$; delaying it by that single decision postpones the same operations to the end of the schedule, and the makespan becomes [113, 133], worse at both endpoints. Run on their own from the empty schedule, Figure 7c and d, SPT and MWKR yield [147, 170] and [143, 161], against a lower bound of [72, 88] set by the total work of J_1 . Neither looks at when a candidate can start, and the decoder appends each operation after the last one on its machine. MWKR, which already departs from the evolved rule at the second decision, dispatches o_{32} before o_{21} and leaves M_1 idle until [50, 56], so that o_{21} , which could have started at time 0, starts at [64, 70]; SPT leaves M_3 idle from [22, 26] to [62, 70] in the same way. The $SLACK^2$ term of the evolved rule penalizes a candidate by how much later it would start than the earliest one.

The example illustrates the mechanism, not its magnitude. Over the first 800 instances generated with the scheme of Section 5.1, removing the

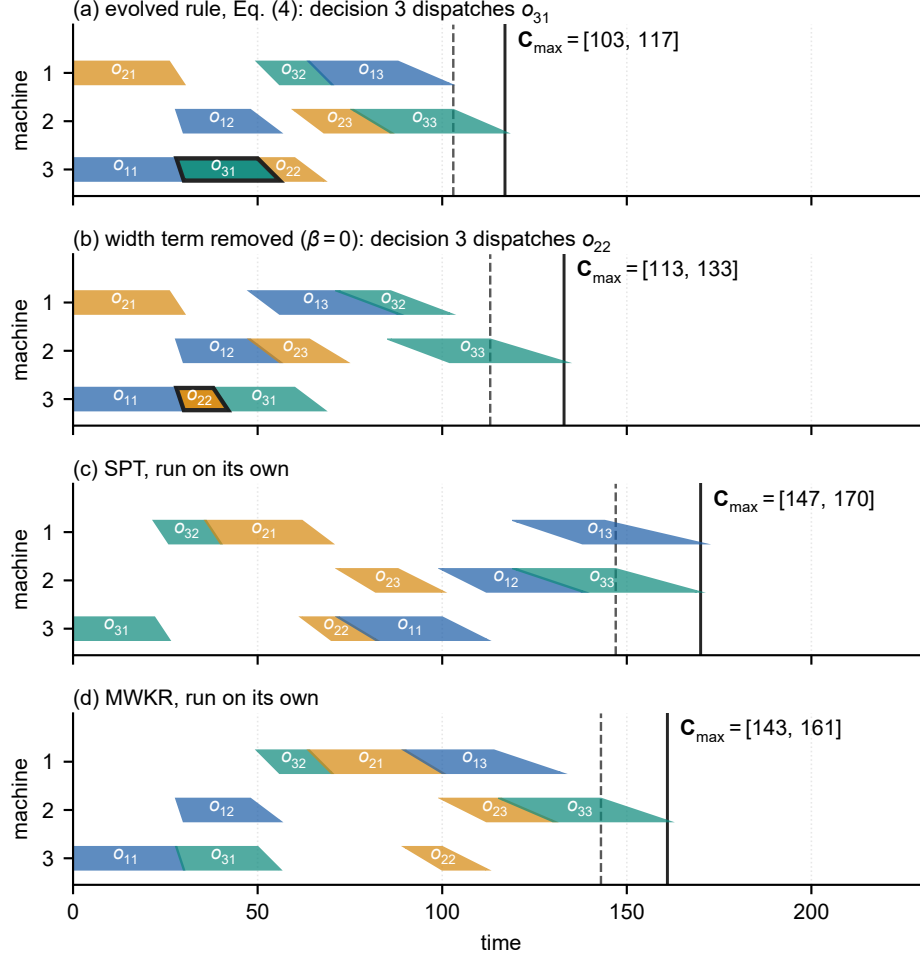


Figure 7: The worked example scheduled by (a) the evolved rule, (b) the same rule without the width term, and (c) SPT and (d) MWKR, each run on its own. In (a) and (b) the outlined operation is the one that separates the two traces.

width term leaves the trace unchanged in 668; of the 132 traces that do change, 93 end at the same makespan, 21 end better with the term and 18 worse. At this scale the width terminals rarely affect a decision, and when they do their effect is barely distinguishable from chance, which is why their

value is established over the 70-instance benchmark in Sections 7.4 and 7.6 rather than here.

7.3. Interval handling and the decoder

Two checks on the comparison with the constructive baselines are reported in Supplementary Section S1. Ranking candidates by the lower endpoint, the midpoint or the upper endpoint of each interval, instead of by the lexicographic order of Eq. (8), moves EST and MWKR by at most 2.4 points, against the 23.3 that separate the best plain rule from the evolved one. And holding the decoder fixed, the evolved rules outperform MWKR under either one: 18.99 against 64.7 in the semi-active decoder and 23.18 against 29.5 inside Giffler and Thompson’s conflict set, where the rules, evolved against the whole eligible set, lose 4.2 points while MWKR gains 35.2. Each method in Table 4 is thus reported with the decoder that suits it, and the margin survives the other choice.

7.4. The value of interval awareness: ablations

The interval-width terminals are the distinctive ingredient of this setting, so we isolate their contribution by ablation: the 30 evolutions are repeated without the width terminals (*PTW*, *ESTW*, *WKRW*), everything else held fixed, including the irace configuration of Section 5.3, which was selected on the full terminal set, so the ablated arm runs with hyperparameters tuned for a larger primitive set than its own. Interval information also enters the evolution through the fitness, which scores every schedule by interval arithmetic. To remove it as well, 30 further rules are evolved on the crisp midpoint counterparts of the training instances (Section 5.1), with the same seeds, configuration and reduced terminal set, so that they never

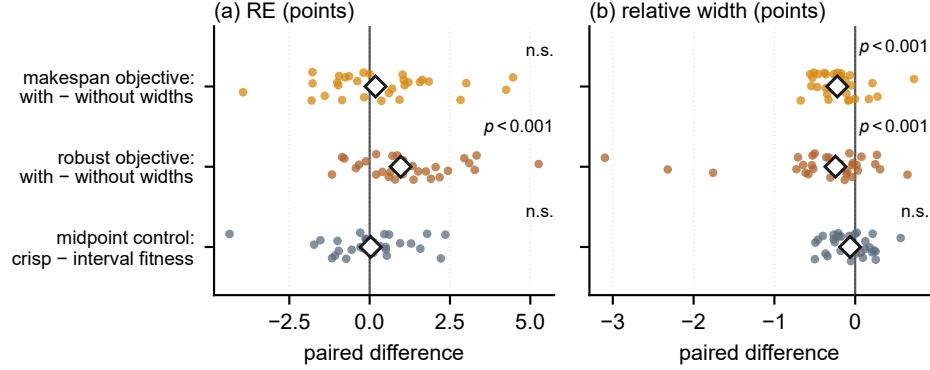


Figure 8: Ablations of the interval machinery, seed by seed over the 30 evolutions of each arm: paired differences in RE (a) and in the relative width of the predicted makespan interval (b) over the 70 instances. Negative values favor the first arm named in each row; diamonds mark the median, and the labels give the paired Wilcoxon test.

handle interval data. Figure 8 compares the arms seed by seed (Section 5.4); Supplementary Table S5 gives their means, and Holm’s correction over its tests changes none of the verdicts.

Under the makespan objective the width terminals do not improve the quantity being optimized: the differences in RE are centered on zero ($z = -0.83$, n.s.), with a sign that favors the smaller terminal set. They do affect the schedules, whose predicted interval is slightly but consistently narrower ($z = 3.40$, $p < 0.001$, $|r| = 0.71$), even though nothing in this objective rewards narrowness. The crisp control is indistinguishable from the interval-trained arm on both measures ($z = -0.13$ and $z = -1.29$, n.s.), although the two fitness functions differ, since max does not commute with taking midpoints. For expected makespan, then, neither the terminals nor the interval fitness is needed. The robust objective of the next section changes this: with the width terminals the predicted interval becomes narrower at a

cost in RE, a trade-off that cannot be replicated on a crisp instance, where the width is identically zero.

7.5. *The robust objective: trading expected makespan for predictability*

The makespan fitness gives the evolution no incentive to trade expected makespan for predictability. To test whether it can make that trade when rewarded for it, we repeat the terminal ablation with a *robust* fitness that explicitly rewards predictability, minimizing

$$f_\lambda = \overline{C}_{\max} + \lambda (\overline{C}_{\max} - \underline{C}_{\max}), \quad (13)$$

that is, a low worst case *and* a narrow interval, with the weight λ setting the balance between the two; we take $\lambda = 1$ here. Under this objective the separation widens. Rules evolved with the width terminals produce schedules whose makespan interval is significantly narrower than those evolved without them (12.04 ± 0.75 versus 12.47 ± 0.20 relative width; paired Wilcoxon $z = 3.22$, $p < 0.001$, $|r| = 0.67$), at the cost of a higher expected makespan (19.62 versus 18.42 RE), which is the trade-off the robust objective is designed to make.

The weight λ of Eq. (13) governs that trade-off, and sweeping it traces the frontier between quality and predictability (Figure 9). Both quantities move monotonically with λ : as the weight grows from 0.5 to 4, the relative width of the makespan interval falls from 12.18 ± 0.50 to 10.54 ± 1.26 while the expected makespan degrades from 19.20 ± 1.52 to 23.27 ± 3.80 RE. At the far end of the sweep the schedules are therefore narrower than any the ablated rules produced, at a cost of about four points of RE.

Read at the level of the arm, which is what a single evolution should be expected to deliver, the price of narrowness rises along the sweep, and

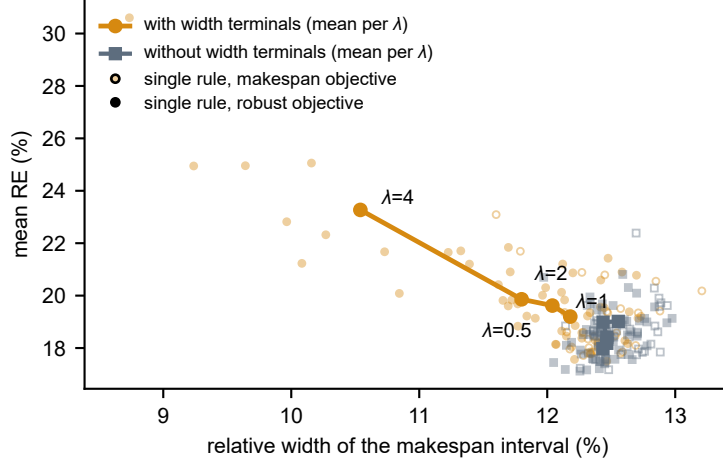


Figure 9: Quality–predictability plane over the 70 instances. Faint markers are individual evolved rules: amber with the interval-width terminals, grey without them; open markers for the makespan objective, filled for the robust one at any λ . Solid curves join the mean of each arm at each λ of the robust-fitness sweep, Eq. (13). With the width terminals, increasing λ trades expected makespan for narrower intervals; without them the five means, $\lambda = 0$ to 4, fall within 0.12 points of width and are left unlabeled.

between $\lambda = 2$ and $\lambda = 4$ the spread between evolutions more than doubles, from 1.74 to 3.80, so the outcome of a single run becomes much less predictable. A weight between 1 and 2 is therefore the default we would recommend. Selecting a rule within an arm amplifies both sides of the trade-off: the narrowest rule of the $\lambda = 1$ arm, used in Section 7.6, reduces the executed deviation by 16.2% at a cost of 7.2 points of RE with respect to the featured rule. A practitioner concerned only with expected makespan can, by Section 7.4, use the smaller terminal set without loss.

Sweeping λ for the ablated arm (ten evolutions per value) shows that it has no such curve: its width stays between 12.43 and 12.56 for every λ from 0 to 4, whereas the full arm goes from 12.18 to 10.54 over the sweep,

Table 8: Predicted quality and executional robustness over the 70 instances: mean RE, relative width of the predicted makespan interval, and $\bar{\varepsilon} (\times 10^3)$ at nominal interval widths and widened by +20% and +40% (Monte Carlo, $K=1000$ realizations, common random numbers across methods). The last column is $|\Delta|$ at nominal widths. Each GP arm is represented by one rule: the makespan arms by their best RE, the robust arms by their narrowest predicted interval, which is what their fitness optimizes.

Method	RE (%)	Width (%)	$\bar{\varepsilon} (\times 10^3)$			$ \Delta $
			+0%	+20%	+40%	
GP, makespan	17.71	12.28	5.70	6.64	7.57	12.30
GP, makespan, no widths	17.17	12.48	5.75	6.80	7.83	12.25
GP, robust $\lambda=1$	24.95	9.24	4.52	5.32	6.08	10.31
GP, robust $\lambda=1$, no widths	17.45	12.05	5.54	6.50	7.42	12.08
GP, robust $\lambda=4$	28.27	8.61	4.21	5.01	5.73	9.78
G&T-MWKR	29.50	13.77	6.29	7.51	8.58	14.94
EST	42.30	13.37	5.66	6.79	7.83	14.64

and no individual rule departs from this band. Penalizing width in the fitness therefore yields narrower intervals only when the rule has terminals through which to act on it: the objective and the representation are not two independent routes to the same end, the first works through the second.

7.6. Robustness under executional uncertainty

We estimate the $\bar{\varepsilon}$ robustness of Eq. (10) by Monte Carlo, drawing $K=1000$ realizations of the durations uniformly on each operation’s interval and executing the fixed processing order on each of them. The sampling measures schedules that are already built, so no distributional assumption enters the problem formulation of Section 3, the interval-arithmetic fitness of Section 4, the evolution or the selection of any rule. The uniform law is

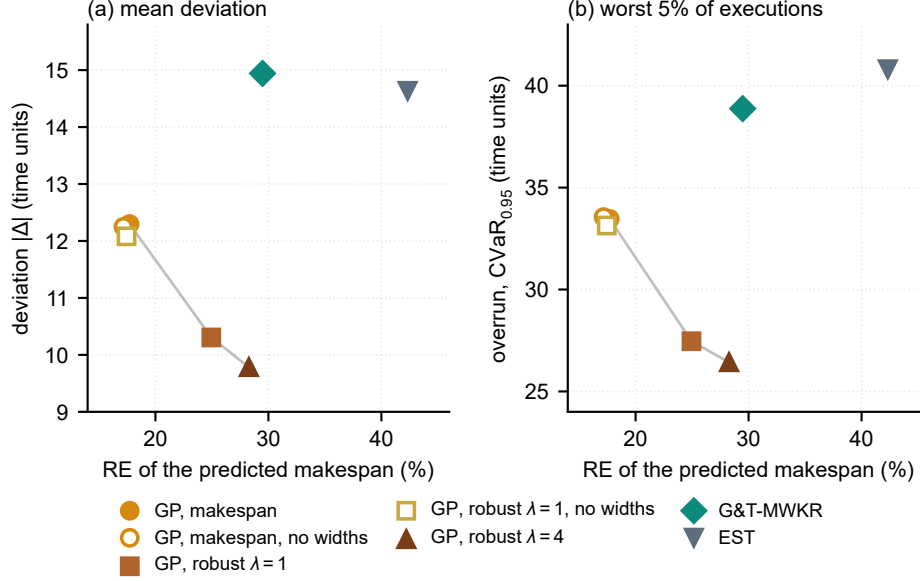


Figure 10: Quality of the prediction against reliability of the execution over the 70 instances, uniform law: (a) mean absolute deviation of the executed makespan from the predicted one, and (b) mean overrun beyond the prediction in the worst 5% of executions, both in time units. Each marker is a method of Table 8; the grey line joins the rules evolved with the width terminals as λ grows.

the least committal choice on a bounded support, and three other laws are considered in Supplementary Section S2. To probe behavior under growing uncertainty we also widen every interval symmetrically about its midpoint by +20% and +40% (clipping negative bounds to zero), re-evaluating the same processing orders.

Figure 10 places each method by the quality of its prediction and the reliability of its execution, the latter in time units so that a larger expected makespan cannot make a method look more robust. Under the makespan objective the width terminals make no difference: the featured rule and its

ablated counterpart coincide in both panels. Both deviate less than G&T-MWKR and EST, 12.30 time units against 14.94 and 14.64, while predicting much shorter makespans. A low deviation does not require a low makespan, so robustness must be optimized for directly.

The robust fitness achieves this, moving the rule down and to the right along the grey line. The narrowest rule of the $\lambda=1$ arm deviates by 10.31 time units, 16.2% less than the makespan rule ($z = -5.55$, $p < 0.001$, $|r| = 0.76$), and the rule at $\lambda=4$ by 9.78, at the cost in expected makespan read on the horizontal axis. The reduction requires the width terminals: evolved under the same fitness without them, the rule stays with the makespan rules. At matched a-priori quality the gain is clearest: the $\lambda=4$ rule and G&T-MWKR predict almost the same makespan, 28.27 against 29.50 points of RE, and the rule deviates 34.5% less, on 69 of the 70 instances ($z = -7.26$, $p < 0.001$, $|r| = 1.00$). Over the worst 5% of executions (Figure 10b) the ordering is the same, and the robust objective reduces the overrun beyond the prediction in about the same proportion as the mean deviation, from 33.46 to 27.46 time units at $\lambda=1$; those executions are nonetheless longer in absolute terms, since their length follows the expected makespan given up for the reliability.

The normalized $\bar{\varepsilon}$ of Table 8 gives the same ordering with one exception: EST appears as robust as the rule, 5.66 against 5.70, because its larger expected makespan inflates the denominator. Supplementary Section S2 reports the paired tests, the per-instance distributions and their growth as the intervals are widened, the comparison over all 30 rules of each arm, where the effects are consistent but of three percent or less, and three other realization laws, under which the ordering and every significant contrast above hold.

7.7. *Asymmetric intervals*

Every instance used so far carries an interval symmetric about its crisp value, so what the width terminals contribute could belong to the generator rather than to the problem. Re-evolved on the right-skewed counterparts of TA11–TA14 described in Section 5.1, with fifteen seeds per arm, the four arms reproduce the main result (Supplementary Section S3): under the robust objective the width terminals narrow the predicted interval from 12.97 to 12.67 and the executed deviation from 13.03 to 12.75, both significant after Holm correction, whereas under the makespan objective they improve neither. The robustness that the width terminals make attainable therefore does not depend on the symmetric generation scheme.

7.8. *Limitations*

Four limitations bound the scope of our conclusions. First, the evolved rules choose from the whole eligible set inside a semi-active decoder. Neither the conflict-set restriction that makes Giffler and Thompson’s procedure yield active schedules, used here only for the baselines, nor an insertion-based generation scheme was placed in the evolutionary loop, and both are known to help weaker constructors. Section 7.3 measures what the first of the two costs after the fact, and the rules lose accuracy: transplanted into the conflict set they were not evolved against, the arm gives up 4.2 points of RE. Whether evolving *inside* that decoder would recover them, or better, is open. Second, all rules were evolved on instances generated with the $\pm 15\%$ symmetric interval scheme standard in the IJSP literature; the robustness analysis of Section 7.6 does re-evaluate the resulting schedules under intervals widened by $+20\%$ and $+40\%$, and Section 7.7 re-evolves the four arms on a right-skewed benchmark, where the robustness contribution of

the width terminals is unchanged, but no rule was evolved under a different *width*, so how far these results carry to uncertainty of a different magnitude remains open. Third, absolute times are specific to our research-grade Python implementation: the times at which the curves of Section 6.5 cross would move with another implementation or language, although the rule and the genetic algorithm are implemented there with the same care; the rankings of the other sections do not depend on it. Finally, no learned policy is among the baselines. The one reinforcement-learning method published for the IJSP (Han et al., 2026) leaves part of the sequencing to be resolved during execution, so it does not belong to the single-pass class compared here, and it reports results on a different selection of classical instances.

8. Conclusions and Future Work

We presented what is, to the best of our knowledge, the first GP hyper-heuristic for job shop scheduling with interval durations: priority rules evolved over a terminal set that exposes both the worst-case values and the widths of the uncertain quantities, compact enough to be read and simplified by hand. The empirical conclusions are four. (i) Evolved rules outperform every deterministic constructive baseline on the 70-instance benchmark by a wide, statistically significant margin (18.99 ± 1.33 over 30 evolutions, 17.71% for the best rule, versus 29.5% for the strongest baseline, which the best rule outperforms on all 70 instances) at the cost of a single constructive pass, and the margin is not an artifact of the schedule generation scheme: holding the decoder fixed, either one, the evolved rules still outperform MWKR. (ii) They generalize zero-shot across all size classes, evolved on 20×15 and best on the unseen 50×15 ; since every terminal is a per-operation attribute

with no size-bound quantities, that transfer requires no retraining or rescaling. (iii) Under the default expected-makespan objective, interval awareness at evolution time is unnecessary: removing the width terminals leaves it unchanged, and evolving on the crisp midpoint instances with scalar fitness does equally well. Under an objective that penalizes interval width, however, an objective with no crisp counterpart, the predicted intervals become substantially narrower and the deviation of the executed makespan from its prediction falls by about a fifth (4.52 against 5.70×10^{-3}); that reduction requires the width terminals, since under the same objective the ablated arm reaches only 5.54 and no value of λ narrows its intervals. The gain comes at the cost of a higher expected makespan, and the weight λ governs the trade-off. It extends to the tail of the distribution: over the worst 5% of executions the overrun beyond the prediction falls in about the same proportion, although those executions remain longer in absolute terms, as the expected makespan given up for the reliability determines. The robustness result survives a change of generator: re-evolved on a right-skewed benchmark, the width terminals again narrow both the predicted interval and the executed deviation under the robust objective, and again leave expected makespan unimproved under the default one.

(iv) Where an evolved rule stands against per-instance search is a question of budget, and it has a measurable answer. Against our Python implementation of the genetic algorithm published for this problem, which shares decoder, evaluator and language with the rule, the rule’s sampled variant is the better use of computation up to a time that grows with the size of the instance, from about 10 s per instance on the classical instances to about 300 s on 50×15 ; beyond it the genetic algorithm ends ahead. The remaining distance to the best published ABC variants is not a matter of budget at

all — a plain population method levels off well short of it — but of the neighborhood structures and local search those methods incorporate. What this paper contributes is therefore a high-quality, interpretable, fast-decision baseline for the IJSP, not a general solver; and alongside a solver it serves to initialize it: seeded with the rule’s permutation, the genetic algorithm is far ahead of its unseeded version during the first seconds, although given enough time the search reaches the same quality without the seed.

Returning to the questions of Section 1: the evolution selects the classical scheduling attributes most often, with the width terminals present in most rules; whether the magnitude of the uncertainty is useful, and whether a robust objective yields more predictable executions, are answered by (iii); and where such rules stand against per-instance search, by (iv).

Future work follows four lines. First, optimizing the numeric weights of an evolved rule, which the sensitivity analysis of Section 7.1 suggests the evolution leaves unfinished. Two properties of this setting shape how such a search should be posed. Because operations are dispatched by *argmin*, the priority is invariant to adding a constant and to multiplying by a positive one, so the standard technique of linear scaling (Keijzer, 2003) cannot help and only the *relative* weights of the additive terms are free, $k - 1$ of them for a rule with k terms. And because the terminal set contains no ephemeral random constants, the evolution must build coefficients structurally, spending nodes of a bounded budget on arithmetic that a constant would express in one. Both an enriched terminal set and a local search over the relative weights, whether applied after evolution or to the elites of each generation, follow from these observations. Second, richer symbolic models: ensembles of complementary rules (Gil-Gala et al., 2021) and co-evolution of construction and tie-breaking. Third, carrying the seeding result of Section 6.5 to

the published metaheuristics themselves: here it is a genetic algorithm of our own that the rule improves, and whether the ABC variants, whose local search starts from a different kind of incumbent, gain as much is open. The pools of diverse sequences produced by GP-best-of- N are directly usable as seeded initial populations. Fourth, a controlled comparison with learned neural constructive policies under matched training and inference budgets, the head-to-head evaluation that the recent literature (Xu et al., 2025) identifies as missing.

CRedit authorship contribution statement

Hernán Díaz: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Resources, Data curation, Writing – original draft, Writing – review & editing, Visualization, Project administration.

Acknowledgements

This work was supported by the Spanish Ministry of Science, Innovation and Universities (MCIN/AEI/10.13039/501100011033) under grant PID2022-141746OB-I00.

Declaration of competing interest

The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The GP hyper-heuristic is implemented from scratch in pure Python/NumPy, with no evolutionary-computation dependencies; evolved rules are stored as plain JSON expression trees and evaluated as drop-in dispatching strategies. All benchmark instances, the 430 evolved rules of every experimental arm, the primary result files behind each table and figure, and a self-contained implementation that re-derives the deposited results from scratch are openly available in a Zenodo repository (Díaz Rodríguez, 2026) (doi:10.5281/zenodo.21716972).

References

- Adams, J., Balas, E., Zawack, D., 1988. The shifting bottleneck procedure for job shop scheduling. *Management Science* 34, 391–401. doi:10.1287/mnsc.34.3.391.
- Afşar, S., Puente, J., Palacios, J.J., González-Rodríguez, I., Vela, C.R., 2025. Solving the energy-aware flexible job shop problem using a memetic algorithm. *Integrated Computer-Aided Engineering* 32, 397–414. doi:10.1177/10692509251353343.
- Allahverdi, A., 2022. A survey of scheduling problems with uncertain interval/bounded processing/setup times. *Journal of Project Management* 7, 255–264. doi:10.5267/j.jpm.2022.3.003.
- Allahverdi, A., Aydilek, H., Aydilek, A., 2014. Single machine scheduling problem with interval processing times to minimize mean weighted completion time. *Computers & Operations Research* 51, 200–207. doi:10.1016/j.cor.2014.06.003.

- Branke, J., Nguyen, S., Pickardt, C.W., Zhang, M., 2016. Automated design of production scheduling heuristics: a review. *IEEE Transactions on Evolutionary Computation* 20, 110–124. doi:10.1109/tevc.2015.2429314.
- Calamita, A., Meloni, C., Pranzo, M., Samà, M., 2025. Fast risk estimation for job shop scheduling solutions under interval uncertainty via machine learning. *Flexible Services and Manufacturing Journal* doi:10.1007/s10696-025-09640-7.
- Carlier, J., 1982. The one-machine sequencing problem. *European Journal of Operational Research* 11, 42–47. doi:10.1016/s0377-2217(82)80007-6.
- Coupvent des Graviers, M.E., Kobrosly, L., Guettier, C., Cazenave, T., 2025. Updating lower and upper bounds for the job-shop scheduling problem test instances. *arXiv preprint arXiv:2504.16106*. doi:10.48550/arXiv.2504.16106.
- Díaz, H., González-Rodríguez, I., Palacios, J.J., Díaz, I., Vela, C.R., 2020. A genetic approach to the job shop scheduling problem with interval uncertainty, in: *Information Processing and Management of Uncertainty in Knowledge-Based Systems (IPMU)*, Springer. pp. 663–676. doi:10.1007/978-3-030-50143-3_52.
- Díaz, H., Palacios, J.J., Díaz, I., Vela, C.R., González-Rodríguez, I., 2022. Robust schedules for tardiness optimization in job shop with interval uncertainty. *Logic Journal of the IGPL* 31, 240–254. doi:10.1093/jigpal/jzac016.
- Díaz, H., Palacios, J.J., González-Rodríguez, I., Vela, C.R., 2023a. An elitist seasonal artificial bee colony algorithm for the interval job shop.

Integrated Computer-Aided Engineering 30, 223–242. doi:10.3233/ica-230705.

Díaz, H., Palacios, J.J., González-Rodríguez, I., Vela, C.R., 2023b. Fast elitist ABC for makespan optimisation in interval JSP. Natural Computing 22, 645–657. doi:10.1007/s11047-023-09953-2.

Díaz Rodríguez, H., 2026. Genetic programming dispatching rules for the interval job shop: instances, evolved rules, results and code. doi:10.5281/zenodo.21716972. [dataset].

Feo, T.A., Resende, M.G.C., 1995. Greedy randomized adaptive search procedures. Journal of Global Optimization 6, 109–133. doi:10.1007/BF01096763.

Fisher, H., Thompson, G.L., 1963. Probabilistic learning combinations of local job-shop scheduling rules, in: Muth, J.F., Thompson, G.L. (Eds.), Industrial Scheduling. Prentice-Hall, Englewood Cliffs, NJ, pp. 225–251.

Giffler, B., Thompson, G.L., 1960. Algorithms for solving production-scheduling problems. Operations Research 8, 487–503. doi:10.1287/opre.8.4.487.

Gil-Gala, F.J., Mencía, C., Sierra, M.R., Varela, R., 2021. Learning ensembles of priority rules for online scheduling by hybrid evolutionary algorithms. Integrated Computer-Aided Engineering 28, 65–80. doi:10.3233/ica-200634.

Gil-Gala, F.J., Đurašević, M., Sierra, M.R., Puente, J., 2025a. Greedy randomised adaptive solution builder using priority rules, in: Proceedings

of the Genetic and Evolutionary Computation Conference Companion (GECCO '25), ACM. pp. 211–214. doi:10.1145/3712255.3726547.

Gil-Gala, F.J., Đurašević, M., Sierra, M.R., Varela, R., 2025b. Tree search hyper-heuristic with application to combinatorial optimization. *Journal of Heuristics* 31, 25. doi:10.1007/s10732-025-09560-7.

Han, P., Liu, Y., Huang, M., 2026. RAPO-RL-TAC: Risk-aware partial-order reinforcement learning with timed automata completion for the interval job shop problem. *Processes* 14, 2995. doi:10.3390/pr14182995.

Haupt, R., 1989. A survey of priority rule-based scheduling. *OR Spektrum* 11, 3–16. doi:10.1007/bf01721162.

Hildebrandt, T., Branke, J., 2015. On using surrogates with genetic programming. *Evolutionary Computation* 23, 343–367. doi:10.1162/evco_a_00133.

Ishibuchi, H., Tanaka, H., 1990. Multiobjective programming in optimization of the interval objective function. *European Journal of Operational Research* 48, 219–225. doi:10.1016/0377-2217(90)90375-L.

Karunakaran, D., Mei, Y., Chen, G., Zhang, M., 2017. Dynamic job shop scheduling under uncertainty using genetic programming, in: *Intelligent and Evolutionary Systems*, Springer. pp. 195–210. doi:10.1007/978-3-319-49049-6_14.

Kazimipour, B., Li, X., Qin, A.K., 2014. A review of population initialization techniques for evolutionary algorithms, in: *2014 IEEE Congress on Evolutionary Computation (CEC)*, IEEE. pp. 2585–2592. doi:10.1109/CEC.2014.6900618.

- Keijzer, M., 2003. Improving symbolic regression with interval arithmetic and linear scaling, in: Genetic Programming (EuroGP), Springer. pp. 70–82. doi:10.1007/3-540-36599-0_7.
- Koza, J.R., 1992. Genetic Programming: On the Programming of Computers by Means of Natural Selection. MIT Press, Cambridge, MA.
- Lawrence, S., 1984. Resource constrained project scheduling: an experimental investigation of heuristic scheduling techniques (Supplement). Technical Report. Graduate School of Industrial Administration, Carnegie-Mellon University. Pittsburgh, PA.
- Lei, D., 2011. Population-based neighborhood search for job shop scheduling with interval processing time. Computers & Industrial Engineering 61, 1200–1208. doi:10.1016/j.cie.2011.07.010.
- Leon, V.J., Wu, S.D., Storer, R.H., 1994. Robustness measures and robust scheduling for job shops. IIE Transactions 26, 32–43. doi:10.1080/07408179408966626.
- López-Ibáñez, M., Dubois-Lacoste, J., Pérez Cáceres, L., Birattari, M., Stützle, T., 2016. The irace package: Iterated racing for automatic algorithm configuration. Operations Research Perspectives 3, 43–58. doi:10.1016/j.orp.2016.09.002.
- Mei, Y., Nguyen, S., Xue, B., Zhang, M., 2017. An efficient feature selection algorithm for evolving job shop scheduling rules with genetic programming. IEEE Transactions on Emerging Topics in Computational Intelligence 1, 339–353. doi:10.1109/TETCI.2017.2743758.

- Moore, R.E., Kearfott, R.B., Cloud, M.J., 2009. Introduction to Interval Analysis. SIAM, Philadelphia, PA. doi:10.1137/1.9780898717716.
- Nguyen, S., Mei, Y., Zhang, M., 2017. Genetic programming for production scheduling: a survey with a unified framework. *Complex & Intelligent Systems* 3, 41–66. doi:10.1007/s40747-017-0036-x.
- Nguyen, S., Zhang, M., Johnston, M., Tan, K.C., 2014. Automatic design of scheduling policies for dynamic multi-objective job shop scheduling via cooperative coevolution genetic programming. *IEEE Transactions on Evolutionary Computation* 18, 193–208. doi:10.1109/TEVC.2013.2248159.
- Palacios, J.J., González-Rodríguez, I., Vela, C.R., Puente, J., 2015. Co-evolutionary makespan optimisation through different ranking methods for the fuzzy flexible job shop. *Fuzzy Sets and Systems* 278, 81–97. doi:10.1016/j.fss.2014.12.003.
- Panwalkar, S.S., Iskander, W., 1977. A survey of scheduling rules. *Operations Research* 25, 45–61. doi:10.1287/opre.25.1.45.
- Park, J., Chun, J., Kim, S.H., Kim, Y., Park, J., 2021. Learning to schedule job-shop problems: representation and policy learning using graph neural network and reinforcement learning. *International Journal of Production Research* 59, 3360–3377. doi:10.1080/00207543.2020.1870013.
- Poli, R., Langdon, W.B., McPhee, N.F., 2008. A Field Guide to Genetic Programming. Lulu Enterprises. Freely available at <http://www.gp-field-guide.org.uk>.
- Sotskov, Y.N., Lai, T.C., Werner, F., 2013. Measures of problem uncertainty

for scheduling with interval processing times. *OR Spectrum* 35, 659–689. doi:10.1007/s00291-012-0306-3.

Sotskov, Y.N., Matsveichuk, N.M., Hatura, V.D., 2020. Schedule execution for two-machine job-shop to minimize makespan with uncertain processing times. *Mathematics* 8, 1314. doi:10.3390/math8081314.

Sprecher, A., Kolisch, R., Drexel, A., 1995. Semi-active, active, and non-delay schedules for the resource-constrained project scheduling problem. *European Journal of Operational Research* 80, 94–102. doi:10.1016/0377-2217(93)e0294-8.

Taillard, E., 1993. Benchmarks for basic scheduling problems. *European Journal of Operational Research* 64, 278–285. doi:10.1016/0377-2217(93)90182-M.

Đurašević, M., Jakobović, D., 2018. A survey of dispatching rules for the dynamic unrelated machines environment. *Expert Systems with Applications* 113, 555–569. doi:10.1016/j.eswa.2018.06.053.

Xu, M., Mei, Y., Zhang, F., Zhang, M., 2025. Learn to optimise for job shop scheduling: a survey with comparison between genetic programming and reinforcement learning. *Artificial Intelligence Review* 58, 160. doi:10.1007/s10462-024-11059-9.

Xu, W., Wu, W., Wang, Y., He, Y., Lei, Z., 2023. Flexible job-shop scheduling method based on interval grey processing time. *Applied Intelligence* 53, 14876–14891. doi:10.1007/s10489-022-04213-9.

Zhang, C., Song, W., Cao, Z., Zhang, J., Tan, P.S., Xu, C., 2020. Learning to dispatch for job shop scheduling via deep reinforcement learning, in:

Advances in Neural Information Processing Systems (NeurIPS), pp. 1621–1632.

Zheng, P., Xiao, S., Zhang, P., Lv, Y., 2024. A two-individual-based evolutionary algorithm for flexible assembly job shop scheduling problem with uncertain interval processing times. *Applied Sciences* 14, 10304. doi:10.3390/app142210304.